

Case Study 8: Smart Education Analytics and Student Performance Prediction System using Apache Spark

A university collects student academic records, attendance data, online learning activities, examination scores, and placement records. The university wants to develop an **Educational Analytics Platform** to improve academic performance and predict student outcomes.

As a Big Data Engineer, design, implement, and deploy the solution using Apache Spark and PySpark.

Q1. Spark Initialization and Data Loading (2 Marks)

Initialize Spark and load educational datasets.

Q2. RDD Implementation (3 Marks)

Create RDDs and perform transformation and action operations.

Q3. Key-Value Operations and Persistence (2 Marks)

Implement key-value operations, shuffle operations, and persistence.

Q4. Spark DataFrame Operations (3 Marks)

Perform DataFrame operations including joins and aggregation.

Q5. Exploratory Data Analysis and Spark SQL (5 Marks)

Perform EDA and execute Spark SQL queries to:

- Analyze student attendance patterns.
- Determine subject-wise performance.
- Identify top-performing students.
- Analyze placement trends.
- Generate semester-wise academic reports.

Q6. ETL Pipeline Development (3 Marks)

Develop ETL pipelines for academic and placement data.

Q7. Machine Learning/Deep Learning Implementation (2 Marks)

Implement a model to predict student performance or placement probability.

Additional Implementation Instructions

A. Version Control using Git and GitHub

- Create a **GitHub repository** for the project.
- Upload all source code, datasets (or dataset links), notebooks, and documentation.
- Maintain proper folder structure and commit history.
- Include a **README.md** file containing project description, installation steps, execution procedure, and output screenshots.

B. Containerization using Docker

- Create a **Dockerfile** for the Spark application.
- Build and test the Docker image.

C. Deployment using Kubernetes

- Create Kubernetes deployment and service YAML files.
- Deploy the application on a Kubernetes cluster (Minikube/Kubernetes/OpenShift).
- Verify successful deployment.

D. DevOps Pipeline Implementation

- Configure a basic CI/CD pipeline using GitHub Actions/Jenkins/GitLab CI.
- Automate build, testing, Docker image creation, and deployment.

E. Project Submission Requirements

Submit:

- Source code
- GitHub repository link
- Dockerfile
- Kubernetes YAML files
- CI/CD pipeline configuration
- Execution screenshots
- EDA and Spark SQL outputs
- Model evaluation report
- Final project documentation